

回顾： 数据处理， 特征工程， 线性回归模型

回归模型

回归模型

- 1 线性正则化模型：岭回归、Lasso、ElasticNet
- 2 强力 boosting：XGBoost 回归
- 3 集成的智慧：Voting 与 Stacking

-
- 普通线性回归（OLS）容易过拟合，且无法处理多重共线性。需要给模型加一点“约束”。

1.15 岭回归 (Ridge Regression)

核心原理：

- 在损失函数中加入 L2 正则化项，降低方差，提高模型的泛化能力。
- 惩罚大系数。

$$\min_w \|X_w - y\|_2^2 + \alpha \|w\|_2^2$$

优点：

- 有效解决多重共线性问题。
- 模型稳定性高，泛化能力强。

缺点：

- 不具备特征选择功能（所有特征都会保留），模型解释性稍弱。

适用场景：

- 特征之间存在高度相关性。
- 特征数量多，但认为所有特征都可能对结果有微小贡献。

总结：像是一个“温和的管家”，它不会开除任何员工（特征），但会限制每个人的工作量（系数），防止有人太累（过拟合）。

1.16 Lasso 回归

核心原理：

- 在损失函数中加入 L1 正则化项。
- 稀疏化：可以将不重要的特征系数直接压缩为 0。

优点：

- 自动特征选择：能生成稀疏模型，提高解释性。
- 适合稀疏模型的求解问题。

缺点：

- 如果特征之间高度相关，它倾向于随机选择一个，而丢弃其他相关的特征（不稳定）。
- 当特征数 > 样本数时，最多只能选 n 个特征。

适用场景：

- 特征数量巨大，且怀疑只有少数特征起作用（稀疏关系）。
- 需要模型具有可解释性（知道哪些特征重要）。

总结：像是一个“严格的裁员经理”，它不仅限制工作量，还会直接开除（系数归零）那些没用的员工（特征）。

$$\min_w \frac{1}{2n_{samples}} \|X_w - y\|_2^2 + \alpha \|w\|_1$$

1.17 ElasticNet 回归

核心原理：

- L1 + L2 的结合体。损失函数 = L1 部分 + L2 部分。
- 通过两个超参数 (α , l1_ratio) 控制两者的比例。

优点：

- 继承了 Lasso 的特征选择和 Ridge 的稳定性。
- 解决了 Lasso 在高度相关特征中“随机选人”的问题（倾向于成组选择相关特征）。

缺点：

- 需要调节两个超参数，计算成本稍高。

适用场景：

- 特征数量 > 样本数量。
- 存在多个高度相关的特征，且希望它们同时被选中或剔除。

总结：“刚柔并济的指挥官”：既有 Lasso 的裁员能力（特征选择），又有 Ridge 的团队协作能力（保留相关特征组）。

$$\min_w \frac{1}{2n_{samples}} \|X_w - y\|_2^2 + \alpha \rho \|w\|_1 + \frac{\alpha(1-\rho)}{2} \|w\|_2^2$$

1.18 XGBoost 回归 (Extreme Gradient Boosting)

核心原理：

- 基于决策树的集成算法（Boosting 思想）。
- 加法模型： 串行建立多棵树，每一棵新树都在修正前一棵树的残差（错误）。
- 引入了二阶泰勒展开和正则化项，效率极高。

优点：

- 精度极高： 通常在结构化数据竞赛中表现最佳。
- 自动处理缺失值、非线性关系。
- 支持并行计算，速度快。

缺点：

- 黑盒模型： 解释性差。
- 参数多，容易过拟合（需要仔细调参）。

适用场景：

- 追求预测精度优先的任务（如 Kaggle 竞赛、金融风控）； 数据包含复杂的非线性关系； 表格型数据。

总结： 像是一个“专家会诊团队”： 第一个医生诊断后， 第二个医生专门看第一个医生没看懂的地方， 第三个医生再看前两个剩下的问题……最后综合所有人的意见。

1.19 Voting 回归模型 (投票回归)

核心原理：

- 并行集成。训练多个不同的基模型（如 Ridge, SVR, Random Forest）。
- 预测策略：对所有基模型的预测结果取平均值或加权平均。

优点：

- 简单易懂，实现方便。
- 通过平均化降低了单一模型的方差，提高稳定性。

缺点：

- 如果所有基模型都犯了同样的错误，Voting 无法纠正。
- 通常不如 Stacking 精度高。

适用场景：

- 基模型之间差异较大（多样性好）。
- 作为集成学习的入门。

总结：“大众评审”：不让一个专家说了算，而是让几个不同背景的专家各自打分，最后取平均分，减少个人偏见。

1.20 Stacking 回归模型 (堆叠泛化)

核心原理：

- 分层集成：第一层 (Base Learners)：训练多个不同的模型，输出预测结果；第二层 (Meta Learner)：将第一层的预测结果作为新特征，输入到一个新的模型（元模型）中进行最终预测。

优点：

- 理论上限最高，元模型可以学习到基模型之间的互补关系。
- 能挖掘出单一模型或简单投票无法发现的规律。

缺点：

- 结构复杂，训练慢。
- 容易过拟合（如果元模型太复杂或数据太少）。
- 代码实现相对复杂（需注意交叉验证防止数据泄漏）。

适用场景：

- 数据挖掘竞赛的最后冲刺阶段。
- 拥有大量数据，且单模型性能遇到瓶颈时。

总结：“导师制团队”：一群初级分析师（第一层）先给出报告，然后一位资深首席分析师（第二层/元模型）阅读所有初级报告，学习如何权衡它们的观点，最后给出终极结论。

1.21 模型比较

模型	特点	优势	适用范围
Ridge	L2 正则，收缩系数	多重共线性，过拟合	特征相关性强，不想丢失任何特征时
Lasso	L1 正则，特征归零	特征选择，高维稀疏	特征极多，想知道哪些特征有用时
ElasticNet	L1+L2，成组选择	Lasso 的不稳定性	特征多且高度相关，需兼顾选择与稳定
XGBoost	迭代纠错，非线性	复杂模式，高精度	默认首选，追求精度，数据是非线性的
Voting	平均融合	降低方差	基模型差异大，想要快速提升稳定性
Stacking	元模型学习	挖掘互补信息	竞赛冲榜，数据量大，需要极致性能